Classical Reasoning Agent

Implementation, Experimental Results, and Limitations

1. Knowledge Representation and Natural Language Parsing

1.1 Representation of Facts and Rules

The reasoning agent represents knowledge using a Horn-clause-style representation.

A fact is stored as a tuple containing a predicate and an argument tuple. For example:

("Kind", ("Bob",))

This represents the fact that Bob is kind.

A rule consists of a body and a head. The body contains the conditions, while the head represents the conclusion that can be derived when those conditions are satisfied.

For example:

(("Kind", ("X",)), ("Young", ("X",))) → ("NotSmart", ("X",))

This represents the rule:

If someone is kind and young, then that person is not smart.

A term is treated as a variable only when it is a single uppercase letter, such as X. Other terms, such as Bob and Bear, are treated as constants. This distinction allows the unification algorithm to distinguish variables from specific entities.

1.2 Natural Language Parsing

The parser converts supported English sentences into the internal symbolic representation using regular-expression (regex) templates. No machine-learning model or large language model is used.

The implemented parser supports selected sentence structures, including:

- Positive and negative facts, such as "Bob is young" and "Bob is not young."
- Transitive verb statements, such as "The bear eats the mouse" and "The bear does not need the mouse."
- Conditional rules, such as "If someone is young, then they are smart."
- Universal rules, such as "Kind, young things are not smart" and "All people who drink coffee are dependent on caffeine."
- Causal statements used in CounterBench, such as "A causes B" and "C and D together cause E."

The parser also recognises five CounterBench question patterns:

1. Would X occur if not Y instead of Y?
2. Would X occur if Y instead of not Y?
3. Would X occur if Y?
4. Would X occur?
5. Does X cause Y?

These question patterns are handled using dedicated regex templates.

2. Reasoning Algorithms

2.1 Unification

Unification determines whether two symbolic atoms can be made identical.

The function match(atom1, atom2, subst) compares their predicates and arguments. If they are compatible, it returns a substitution dictionary. Otherwise, it returns None.

For example:

Kind(X) and Kind(Bob)

can be unified using the substitution:

X = Bob

This allows general rules to be applied to specific facts.

2.2 Forward Chaining

Forward chaining is the primary inference algorithm used by the agent.

The algorithm starts with the available facts and repeatedly checks whether the conditions of any rule are satisfied.

The process is:

1. Initialise the fact set with the known facts.
2. Select a rule and attempt to match its body against the available facts.
3. Apply the resulting substitutions to the rule head.
4. Add newly derived facts to the fact set.
5. Repeat until no new facts can be derived or the maximum limit of 300 iterations is reached.

The final fact set is then used to evaluate the query.

2.3 Forward Chaining with a Blocked Set

A blocked set is used for the implemented CounterBench counterfactual questions.

It prevents selected atoms from being used or derived during inference. The reasoner performs forward chaining under the modified conditions and checks whether the target event can still be derived.

For example, if an event is blocked, the reasoner checks whether the target event remains reachable without that event.

This provides a simple mechanism for evaluating the counterfactual question patterns supported by the implementation.

2.4 Other Techniques

The final implementation primarily uses unification and forward chaining.

It does not use backward chaining, an external logical solver, Prolog, generative AI, or lookup tables containing dataset-specific answers.

3. Experiments on Public Datasets

The reasoning agent was evaluated on the complete public training splits of RuleTaker, CounterBench, and FOLIO.

The accuracy was calculated as:

Accuracy = (Number of Correct Predictions / Total Number of Examples) × 100

The experimental results are presented below.

Dataset	Accuracy	Correct Predictions	Total Examples
RuleTaker	70.04%	336,293	480,152
CounterBench	64.50%	645	1,000
FOLIO	61.34%	614	1,001

Inference time was measured separately for each dataset during execution. The notebook prints the measured time for each evaluation.

3.1 RuleTaker

The agent achieved 70.04% accuracy on the complete RuleTaker training split.

The parser supports several basic logical sentence structures, including positive facts, negative facts, transitive relations, and conditional rules.

However, its regex-based design limits the range of sentences it can interpret correctly.

Some unsupported or insufficiently handled structures include:

- Disjunctions, such as "X is young or Y is kind."
- Relative clauses, such as "The bear that eats the mouse is smart."
- More complex sentence structures containing multiple embedded clauses.

When these structures are not correctly parsed, the resulting symbolic representation may be incomplete or incorrect. This affects subsequent inference even when the forward-chaining algorithm itself is functioning correctly.

The result indicates that the reasoning mechanism can derive conclusions from supported logical structures, but the natural-language parser limits overall performance.

3.2 CounterBench

The agent achieved 64.50% accuracy on the complete CounterBench training split.

The implementation converts supported causal statements into symbolic rules. Statements such as "A causes B" and "A and B together cause C" are represented using single-body and multi-body rules.

The blocked-set mechanism is used to evaluate counterfactual questions by modifying the initial conditions and checking whether the target event can still be derived.

Although the implementation supports the five identified CounterBench question patterns, the overall accuracy indicates that the current approach does not handle all examples correctly.

Possible sources of errors include limitations in causal sentence parsing, assumptions about source atoms, and the simplified treatment of counterfactual conditions.

Therefore, the result demonstrates partial coverage of CounterBench rather than complete causal reasoning capability.

3.3 FOLIO

The agent achieved 61.34% accuracy on the complete FOLIO training split.

FOLIO contains complex natural-language statements involving first-order logic, negation, disjunction, quantifiers, and embedded clauses.

The current regex-based parser does not adequately support many of these structures.

For example, a sentence involving multiple clauses connected by "or" may not be converted into the correct logical representation. When the parser fails to identify the intended structure, the resulting atom may not match the facts or rules in the knowledge base.

Consequently, the reasoning engine cannot reliably perform the required inference.

The 61.34% result should therefore be interpreted cautiously. It does not establish that the agent can perform general first-order logical reasoning on FOLIO.

4. Examples of Reasoning Traces

The following examples illustrate the internal reasoning process for sentence structures supported by the implementation.

Trace A: RuleTaker — Facts and a Universal Rule

Context:

Bob is kind. Bob is young. Kind, young things are not smart.

Query:

Bob is not smart.

Initial facts:

- Kind(Bob)
- Young(Bob)

Rule:

Kind(X) AND Young(X) → NotSmart(X)

Reasoning process:

1. The reasoner checks the rule body against the available facts.
2. Kind(X) matches Kind(Bob), giving the substitution X = Bob.
3. Young(X) also matches Young(Bob).
4. Both conditions are satisfied.
5. The rule derives NotSmart(Bob).

Result: True

Trace B: RuleTaker — Transitive Verb Fact

Context:

The bear eats the mouse. The bear does not need the mouse.

Query:

The bear eats the mouse.

Parsed facts:

- Eat(Bear, Mouse)
- NotNeed(Bear, Mouse)

Reasoning process:

The query is converted into the atom Eat(Bear, Mouse). This atom is already present in the initial fact set.

Result: True

Trace C: CounterBench — Counterfactual Reasoning

Context:

We know that Ziklo causes Blaf, Blaf and Trune together cause Vork, and Vork causes Lumbo.

Query:

Would Lumbo occur if not Ziklo instead of Ziklo?

Parsed rules:

- Ziklo → Blaf
- Blaf AND Trune → Vork
- Vork → Lumbo

Initial source atoms:

- Ziklo
- Trune

These are identified as source atoms because they occur in rule bodies but do not occur as rule heads.

Counterfactual modification:

- Blocked atom: Ziklo
- Remaining source atom: Trune

Reasoning process:

1. Forward chaining starts with Trune.
2. Ziklo cannot be used because it is blocked.
3. Therefore, Blaf cannot be derived.
4. Since both Blaf and Trune are required, Vork cannot be derived.
5. Consequently, Lumbo cannot be derived.

Result: False

5. Limitations and Future Improvements

The main limitation of the current reasoning agent is its regex-based natural-language parser.

The parser supports a limited collection of sentence structures and does not provide general-purpose natural-language understanding. Unsupported sentence constructions can produce incomplete or incorrect symbolic representations, which directly affects reasoning accuracy.

The major limitations are:

1. Limited support for disjunctions and embedded clauses.
2. Inadequate handling of complex natural-language negation.
3. Limited support for relative clauses and complex sentence structures.
4. Dependence on predefined regex templates.
5. Incomplete coverage of the logical and linguistic structures present in the evaluated datasets.
6. Simplified treatment of counterfactual reasoning.

The implementation uses forward chaining as its primary inference mechanism. Backward chaining was removed from an earlier version to reduce implementation complexity. The current version therefore uses a simpler inference approach, but this does not eliminate the limitations introduced by natural-language parsing and knowledge representation.

Future improvements could include:

- Replacing regex templates with a more structured grammar-based parser.
- Improving support for negation, disjunction, quantifiers, and relative clauses.
- Extending the internal representation to handle more complex logical expressions.
- Improving the treatment of causal dependencies and counterfactual conditions.
- Adding more comprehensive tests for unification and inference.
- Evaluating the system on unseen examples to measure its generalisation ability.

A more capable parser and a more expressive logical representation could improve the overall performance without requiring a complete replacement of the existing forward-chaining mechanism.

6. Conclusion

The implemented Classical Reasoning Agent converts selected natural-language statements into symbolic facts and rules and uses unification and forward chaining to answer logical queries.

The experiments on the complete public training splits resulted in accuracies of 70.04% on RuleTaker, 64.50% on CounterBench, and 61.34% on FOLIO.

The results demonstrate that the agent can perform symbolic inference on supported sentence structures. However, its performance is limited by the regex-based parser, which cannot adequately represent many complex natural-language constructions.

The implementation provides a basic classical reasoning framework without relying on generative AI or dataset-specific answer lookup. Further improvements in parsing, logical representation, and inference are required to make it more reliable on diverse and previously unseen reasoning problems.